

PDM Professional Web API (REST) — Examples

(1/1): GettingStarted-Welcome

SOLIDWORKS API Documentation (converted from CHM)

GettingStarted (Topic)

PDM Pro Web API Help

Getting Started

Glossary Item Box

- [Prerequisites](#)
- [Background](#)
- [Javascript, Python, and C# Console Examples](#)
- [Install and Enable CORS](#)
- [Event hooks](#)

Prerequisites

In order to build applications that use the operations in the SOLIDWORKS PDM Professional Web API, you need:

- **SOLIDWORKS PDM Professional**

Read the **Installation Guide.pdf** in **c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS PDM\Lang\lang**. Follow the outline in **Chapter 2 Installation Overview** and all the instructions in ensuing chapters to install required and optional (SOLIDWORKS PDM Professional only) components.

- **SOLIDWORKS PDM Web API Server (SOLIDWORKS PDM Professional only)**

In the installation guide, follow the instructions under **Chapter 5 Installing SOLIDWORKS PDM - Installing SOLIDWORKS PDM Web API Server**. At <https://help.solidworks.com> select a version and a language and navigate to **SOLIDWORKS PDM > SOLIDWORKS PDM Administration Tool > Administering Web API Servers (For SOLIDWORKS PDM Professional only)**. Using the PDM Administration Tool, you must add the web API server, add a vault to the web API server, and configure the vault.

- **Configure IIS Application Pools**

If you are using your computer as both the web API server and the web client, in order to have permission to stage files for check-in you may need to change the Application Pool entity of your web server to be your computer login.

1. In IIS under your local host in the Connections list, click **Application Pools**.
2. Right-click SOLIDWORKS PDM WebAPI and select **Advanced Settings**.
3. Change **Process Model > Identity** to your computer login.
4. Click **OK**.

Read more about IIS configuration at <https://learn.microsoft.com/en-us/iis>.

- **Cross-origin resource sharing (CORS)**

If you are not familiar with CORS, then please do an internet search for "Cross-origin resource sharing". CORS is required if you use browser-based programming languages like Java, PHP, Javascript, Python, NodeJs etc. or access the PDM Pro Web API from a different domain. If Internet Information Services (IIS) is not already active, do an internet search of "how to open internet information services", and follow the instructions. Then [install CORS](#) and configure the SWPDMWebAPI web site in IIS to enable CORS. See the Javascript and Python examples in [api/{vaultName}/authenticate](#).

- **Postman**

Some examples in this help demonstrate usage of PDM Pro Web API endpoints using Postman collections. Each collection is a typical use case involving multiple endpoints. Postman provides an easy interface for you to verify HTTP requests using the PDM Pro Web API. Once you are satisfied that the API is working in Postman, you can confidently develop your own applications in any language. You can export the HTTP requests you set up in Postman to another language using the **Code (</>)** button in the right sidebar of Postman.

Download and install Postman. Use an internet search engine to locate this free software. <https://www.postman.com/downloads/>

Click and save the following environment collection to your computer. Then drag/drop it into Postman:

[WebAPI.postman_environment.json](#)

Click **Environment Quick Look** next to the WebAPI enviroment in Postman to edit the following global variables (and possibly others) to work with your installation:

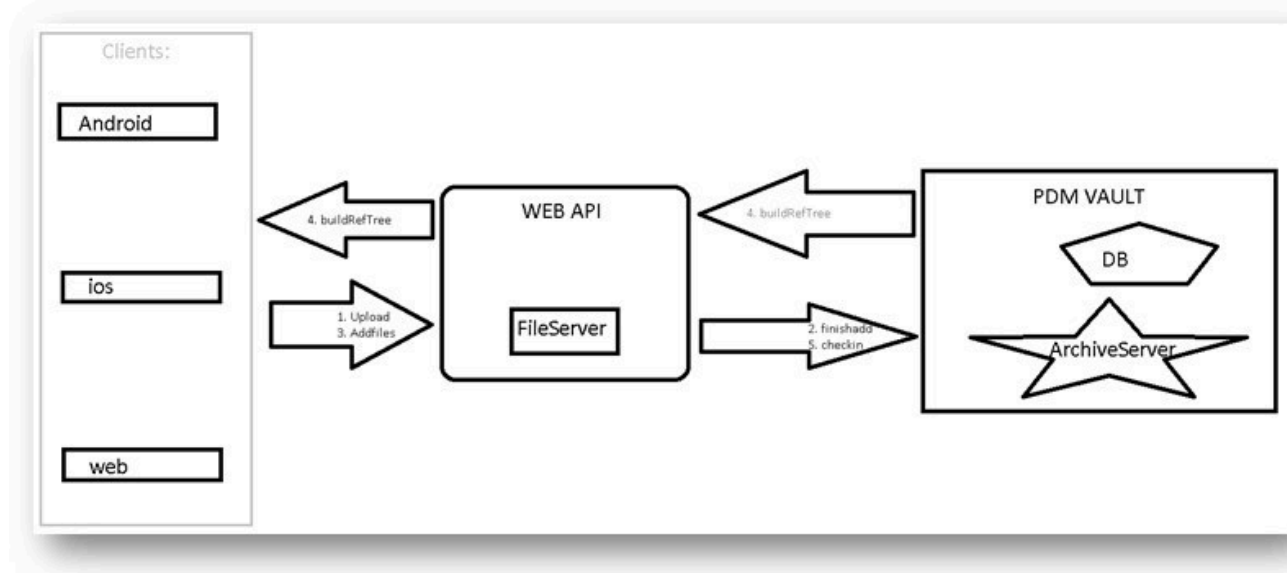
- host
- vaultName
- UserName
- Password
- basicURL
- token
- jsonTypeHeader

Look for links to other Postman collections in this help that you can drag/drop into Postman. The collections all contain an authenticate HTTP request that you must send to obtain a bearer token to send with all other HTTP requests.

- **Microsoft Visual Studio (for C# and ASP.NET)**

You can use a .NET project template to create a C# Console App (.NET Core) or an ASP.NET Web Application (.NET Framework) that accesses the PDM Pro Web API. See the C# console application example in [api/{vaultName}/authenticate](#).

Background



The SOLIDWORKS PDM Professional Web API is an extension of Microsoft's [ASP.NET Web API](#) which is a framework that allows you to create Representational State Transfer (REST) applications on the .NET Framework.

If you are not familiar with REST, then please do an internet search of:

- Representational state transfer (REST)
- ASP.NET REST APIs with .NET and C#

Hypertext Transfer Protocol (HTTP) enables communication between clients and servers through a request-response protocol.

If you are not familiar with HTTP, then please do an internet search of:

- HTTP Methods

Your web service application can authenticate logged-in users and set up [JSON Web Tokens \(JWT\)](#) Bearer tokens to securely authorize and restrict API access. You must obtain a bearer token to send with most HTTP requests of the PDM Pro Web API.

Read about the importance of [API Security](#).

Javascript, Python, and C# Console Examples

You will find examples scattered throughout the help. To get started with authenticating users, authorizing access, and using the GET and POST HTTP methods, follow the examples in [api/{vaultName}/authenticate](#).

Install and Enable CORS

If you use browser-based programming languages like Java, PHP, Javascript, Python, NodeJs etc., you need CORS. The CORS extension can be downloaded and installed from Microsoft and enabled in IIS by editing the WebAPI **web.config** file as follows:

1. Go to <https://www.iis.net/downloads/microsoft/iis-cors-module>. Download and install the CORS module.
2. Navigate to the PDM Pro Web API installation location (e.g., **C:\inetpub\wwwroot\SOLIDWORKSPDM\WebApi**).
3. Open **web.config** in any text editor with Administrator privileges (Notepad or Notepad++, for example).
4. Add the following lines to the `<configuration><system.webServer>` section:

```
<cors enabled='true'>
  <add origin='*'>
    <allowHeaders allowAllRequestedHeaders='true' />
    <allowMethods>
      <add method='GET' />
      <add method='POST' />
      <add method='PUT' />
      <add method='DELETE' />
    </allowMethods>
  </add>
</cors>
```

The `<system.webServer>` section should look like this:

```
<system.webServer>
  <security>
    <requestFiltering>
      <requestLimits maxAllowedContentLength='4294967295' />
    </requestFiltering>
  </security>
  <handlers>
    <remove name='ExtensionlessUrlHandler-Integrated-4.0' />
    <remove name='OPTIONSVerbHandler' />
    <remove name='TRACEVerbHandler' />
    <add name='Owin' verb='*' path='*' type='Microsoft.Owin.Host.SystemWeb.OwinHttpHandler, Microsoft.Owin.Host.SystemWeb' />
    <add name='ExtensionlessUrlHandler-Integrated-4.0' path='*' verb='*' type='System.Web.Handlers.TransferRequestHandler'
preCondition='integratedMode, runtimeVersionv4.0' />
  </handlers>
  <cors enabled='true'>
    <add origin='*'>
```

```
<allowHeaders allowAllRequestedHeaders='true' />
<allowMethods>
  <add method='GET' />
  <add method='POST' />
  <add method='PUT' />
  <add method='DELETE' />
</allowMethods>
</add>
</cors>
</system.webServer>
```

5. Save **web.config**.
6. Open the IIS Manager.
7. Stop the SWPDMWebAPI site.
8. Stop the SOLIDWORKS PDM WebAPI application pool.
9. Restart the SOLIDWORKS PDM WebAPI application pool.
10. Restart the SWPDMWebAPI site.

Event hooks

As of 2025, PDM Pro Web API provides support for configuring and handling vault events and managing custom hook add-in DLLs. Click and save the following JSON collections to your computer. Then drag/drop them into Postman:

[HooksManagement.postman_collection.json](#)

[GetPostFileServerRoot.postman_collection.json](#)

For details, see the [Web Event Hooks Programming Guide](#).

This example authenticates credentials that you enter in a form.

1. Read the **Getting Started** topic.
2. Using the following form, enter the machine name where your database is hosted, port, vault name, and login credentials.
3. Examine the Response text area.

NOTES:

- Errors are also printed to Response.
 - To recreate this example:
 1. Right-click in this page and select **View source**.
 2. Save the file as an HTML file.
 3. Open the HTML file in your web browser.
 4. If using Windows Internet Explorer and running scripts is blocked, click the Information bar near the top of the webpage and select **Allow Blocked Content**.
-

ModifyDatacard (Topic)

This example uses JSON objects. Do not try to run this example from the compiled help (chm). The browser that underpins compiled help is Microsoft Internet Explorer, which does not recognize JSON objects. To recreate this example in a supported web browser:

1. Right-click in this page and select **View source**.
2. Save the file as an HTML file.
3. Open the HTML file in a supported web browser (Microsoft Edge, Google Chrome, or Mozilla Firefox).

This example authenticates credentials that you enter in a form, checks out a SOLIDWORKS file, modifies the file's datacard, creates a changeset, adds the modified file to the changeset, and checks in the changeset.

1. Follow the **Getting Started** instructions.
2. Ensure that a Quick Start data card exists. (A Quick Start data card for SOLIDWORKS files is optionally created during vault setup.)
3. Ensure that at least one SOLIDWORKS file is checked into the root folder (folder Id = 1) of your vault.
4. In the form below, enter the machine name where your web server is hosted, your web server's application port, a PDM Pro vault name, and your PDM Pro Admin credentials.
5. Click **Authenticate**.
6. Examine the Response text area for the JSON web token.
7. Click **Find IDs** to get the IDs of all files in the root folder of your vault.
8. Examine the Response text area and locate one SOLIDWORKS part's ID, version, and folder ID.
9. Enter the file's version in the Version field, file ID in the FileID field, and the root folder ID in the FolderID field.
10. Click **Check out SOLIDWORKS file**.
11. Click **Display the file's data card**.
12. Examine the Response text area for file's data card values.
13. Inspect the first variable (Project Name). Note the VarId (48). Enter the 48 in the ID of Project Name variable field. Click **Change variable value** to change the Project Name value to PDM Pro Web API.
14. Examine the Response text area.
15. Click **Create change set**.
16. Examine the Response text area for the changeset ID. Enter the changeset ID in the Changeset ID field.
17. Click **Add files to changeset**.
18. Click **Check in file**.

NOTES:

- Errors are also printed to the Response text area.
-

PDM Pro API Web Service

Description

The PDM Professional API Web Service consists GET, POST, PUT, PATCH, and DELETE HTTP requests that you call from your web client application. You must send the authenticate request before all others. See [Getting Started](#).

Authentication

BOM

GET	api/{vaultName}/bom/{bomTypeId}/{fileId}/{version}/{folderId}/computed?configId={configId}&latest={latest} GET: api/{VaultName}/bom/{BomTypeId}/{FileID}/{Version}/{FolderID}/weldmentcutlist or api/{VaultName}/bom/{BomTypeId}/{FileID}/{Version}/{FolderID}/computed Get computed BOM Gets the computed BOM.
GET	api/{vaultName}/bom/{bomTypeId}/{fileId}/{version}/{folderId}/weldmentcutlist?configId={configId}&latest={latest} GET: api/{VaultName}/bom/{BomTypeId}/{FileID}/{Version}/{FolderID}/weldmentcutlist or api/{VaultName}/bom/{BomTypeId}/{FileID}/{Version}/{FolderID}/computed Get computed BOM Gets the weldment cut list BOM.

Configuration

PUT	api/{vaultName}/configuration/hooks/dll Puts add-in hook DLLs in the specified vault.
DELETE	api/{vaultName}/configuration/hooks/dll Deletes add-in hook DLLs from the specified vault.
GET	api/{vaultName}/configuration/hooks/url Retrieves the list of webhooks configured in the specified vault.
POST	api/{vaultName}/configuration/hooks/url Sets the webhooks for the specified vault.
GET	api/configuration/fileserverroot GET: api/configuration/fileserverroot Get file server root path(WebAPI storage)Gets the file server root path.
POST	api/configuration/fileserverroot POST: api/configuration/fileserverroot Add file server root pathMigrates add-ins from previous file server root to the specified new one and reloads hooks instances.

GET	api/configuration/vaults?vaultName={vaultName} GET: api/configuration/vaults Get all vaults from the config file Gets all vaults from the vault configuration file.
-----	---

Data Card

GET	api/{vaultName}/files/{fileId}/{version}/datacard?folderId={folderId} GET: api/{VaultName}/files/{FileID}/{Version}/datacard Get file data card Gets the data card for the specified file, file version, and vault.
POST	api/{vaultName}/files/{fileId}/datacard?folderId={folderId} Saves the specified data card variables to the specified file.
GET	api/{vaultName}/folders/{folderId}/datacard GET: api/{VaultName}/folders/{FolderID}/datacard Get folder data card Gets the data card for the specified folder and vault.

Default

POST	api/{uri} Uses the specified URI to post information.
PUT	api/{uri} Uses the specified URI to save information.
PATCH	api/{uri} Uses the specified URI to patch information.
DELETE	api/{uri} Uses the specified URI to delete information.

File

GET	api/{vaultName}/files/{fileId}/{folderId}/history Gets a specified file's history.
GET	api/{vaultName}/files/{fileId}/{folderId}/moves Gets the history of moves for the specified file.
GET	api/{vaultName}/files/{fileId}/{folderId}/versions GET: api/{VaultName}/files/{FileID}/versions Get file version info

	Gets the specified file's version information.
DELETE	api/{vaultName}/files/{fileId}/{folderId}?destroy={destroy} Deletes the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{configId}/{folderId}/whereused?anyVersion={anyVersion} GET: api/{VaultName}/files/{FileID}/{Version}/whereused Get file where usedGets the parent files where the specified file is used.
GET	api/{vaultName}/files/{fileId}/{version}/{configId}/whereused?folderId={folderId}&anyVersion={anyVersion} GET: api/{VaultName}/files/{FileID}/{Version}/whereused Get file where usedGets the parent files where the specified file is used.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId} Gets information about the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/ActiveConfig Gets the active configuration for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/allreferences?configId={configId}&includeSubParents={includeSubParents} GET: api/{VaultName}/files/{FileID}/{Version}/allreferences Get all file references Gets the reference tree of the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/configurations Gets the configurations for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/download Gets the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/info Gets information about the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/info/extended Gets extended version information for a specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/references?configId={configId} GET: api/{VaultName}/files/{FileID}/{Version}/references Get file references Gets the top level file references of the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/thumbnails Gets a specified file's thumbnail.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/transitions Gets the file transitions for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/{folderId}/variables Gets configuration variable information for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/ActiveConfig?folderId={folderId} Gets the active configuration for the specified file.

GET	api/{vaultName}/files/{fileId}/{version}/allreferences?configId={configId}&folderId={folderId}&includeSubParents={includeSubParents} GET: api/{VaultName}/files/{FileID}/{Version}/allreferences Get all file references Gets the reference tree of the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/configurations?folderId={folderId} Gets the configurations for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/download?folderId={folderId} Gets the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/info/extended?folderId={folderId} Gets extended version information for a specified file.
GET	api/{vaultName}/files/{fileId}/{version}/info?folderId={folderId} Gets information about the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/references?configId={configId}&folderId={folderId} GET: api/{VaultName}/files/{FileID}/{Version}/references Get file references Gets the top level file references of the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/thumbnails?folderId={folderId} Gets a specified file's thumbnail.
GET	api/{vaultName}/files/{fileId}/{version}/transitions?folderId={folderId} Gets the file transitions for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/variables?folderId={folderId} Gets configuration variable information for the specified file.
GET	api/{vaultName}/files/{fileId}/{version}/whereused?configId={configId}&folderId={folderId}&anyVersion={anyVersion} GET: api/{VaultName}/files/{FileID}/{Version}/whereused Get file where used Gets the parents of the specified file.
GET	api/{vaultName}/files/{fileId}/{version}?folderId={folderId} Gets information about the specified file.
GET	api/{vaultName}/files/{fileId}/bominfo?folderId={folderId} GET: api/{VaultName}/files/{FileID}/bominfo Get file BOM info Gets the BOM information for the specified file.
GET	api/{vaultName}/files/{fileId}/history?folderId={folderId} Gets a specified file's history.
GET	api/{vaultName}/files/{fileId}/info?version={version}&folderId={folderId} Gets information about the specified file.
GET	api/{vaultName}/files/{fileId}/moves?folderId={folderId}

	Gets the history of moves for the specified file.
GET	api/{vaultName}/files/{fileId}/transitions?version={version}&folderId={folderId} Gets the file transitions for the specified file.
GET	api/{vaultName}/files/{fileId}/versions?folderId={folderId} GET: api/{VaultName}/files/{FileID}/versions Get file version info Gets the specified file's version information.
DELETE	api/{vaultName}/files/{fileId}?folderId={folderId}&destroy={destroy} Deletes the specified file.
GET	api/{vaultName}/files/{fileId}?version={version}&folderId={folderId} Gets information about the specified file.
POST	api/{vaultName}/files/buildtree/checkout POST: api/{VaultName}/files/buildtree/checkout Build checkout references tree Gets the tree of references in the specified file's checkout.
POST	api/{vaultName}/files/buildtree/undo POST: api/{VaultName}/files/buildtree/undo Build undo checkout references tree Builds the reference tree to undo the checkout of the specified file.
POST	api/{vaultName}/files/CheckOut POST: api/{VaultName}/files/CheckOut Lock file Checks out the specified file(s).
POST	api/{vaultName}/files/info POST: api/{VaultName}/files/info Gets information about the specified files.
POST	api/{vaultName}/files/infofrompath Gets information about a file in the specified vault.
POST	api/{vaultName}/files/UndoCheckOut POST: api/{VaultName}/files/UndoCheckOut Undo lock file Unlocks the specified file.

Folder

GET	api/{vaultName}/folders/{folderId}/browse GET: api/{VaultName}/folders/{FolderID}/browse Get browse folder Gets the files and folders in the specified folder.
-----	--

GET	api/{vaultName}/folders/{folderId}/info GET: api/{VaultName}/folders/{FolderID} or api/{VaultName}/folders/{FolderID}/info Get folder info Gets information for the specified folder.
DELETE	api/{vaultName}/folders/{folderId}?destroy={destroy} DELETE: api/{VaultName}/folders/{FolderID} Delete folder Deletes the specified folder.
PUT	api/{vaultName}/folders/{parentFolderId} PUT: api/{VaultName}/folders/{ParentFolderID} Add folder Adds a folder to the specified folder.
POST	api/{vaultName}/folders/info POST: api/{VaultName}/folders/info Get folder info Gets information about specified folders in the specified vault.

Group

GET	api/{vaultName}/groups/{groupId} GET: api/{VaultName}/groups/{GroupID} or api/{VaultName}/groups/{GroupID}/info Get group info Gets information about the specified group.
GET	api/{vaultName}/groups/{groupId}/info GET: api/{VaultName}/groups/{GroupID} or api/{VaultName}/groups/{GroupID}/info Get group info Gets information about the specified group.
GET	api/{vaultName}/groups/all GET: api/{VaultName}/groups or api/{VaultName}/groups/all Get groups Gets the groups in the specified vault.

Notification

POST	api/{vaultName}/notifications/markRead POST: api/{vaultName}/notifications/markRead Mark message as read Marks a notification as read.
GET	api/{vaultName}/notifications?all={all}&pageNo={pageNo}&pageSize={pageSize} GET: api/{VaultName}/notifications Get user notifications Gets the user notifications.

Progress

GET	api/{vaultName}/progress/{guid}/status GET: api/{vaultName}/progress/{guid}/status Get operation status Gets the status of the specified operation.
-----	---

Stage

PUT	api/{vaultName}/checkin/{changesetId}/{overrideVersion} PUT: api/{vaultName}/checkin/{changesetId}/{overrideVersion=false} Check in files from the changeset Checks in the specified changeset.
PUT	api/{vaultName}/checkin/addfiles/{changesetId} PUT: api/{vaultName}/checkin/addfiles/{changesetId=0} Upload files or document ids to changeset to the check in operation Adds files to a changeset for checkin. (See Remarks)
GET	api/{vaultName}/checkin/buildtree/{changesetId} GET: api/{vaultName}/checkin/buildtree/{changesetId} Get references tree Builds a reference tree of files that are uploaded to the specified changeset.
PUT	api/{vaultName}/files/{changesetId}/finishadd PUT: api/{vaultName}/files/{changesetId}/finishadd Finish of add file operation. This method adds files from changeset to the vault. Adds files in the specified changeset to the vault. (See Remarks)
PUT	api/{vaultName}/files/{changesetId}/upload PUT: api/{vaultName}/files/{changesetId}/upload Upload file to changeset to add files operations Uploads files to the specified changeset. (See Remarks)
PUT	api/stage/{vaultName}/{documentId}?folderId={folderId}&overrideVersion={overrideVersion} Checks in the specified document.

State Transition

GET	api/{vaultName}/state/{documentId}/{folderId}/transitions Gets the transitions available for the specified file.
GET	api/{vaultName}/state/{documentId}/transitions?folderId={folderId} Gets the transitions available for the specified file.
GET	api/{vaultName}/state/{transitionId} GET: api/{vaultName}/state/{transitionId} Get transition info

	Gets information about the specified transition.
POST	api/{vaultName}/state/{transitionId}/changestate?revoke={revoke} POST: api/{vaultName}/state/{transitionId}/changestate Change state Changes the state of the specified documents.
POST	api/{vaultName}/state/{transitionId}/DynamicNotificationUsers POST: api/{vaultName}/state/{transitionId}/DynamicNotificationUsers Get dynamic notification users Gets the users to notify when the specified items undergo the specified transition.
POST	api/{vaultName}/state/{transitionId}/HistoryComments POST: api/{vaultName}/state/{transitionId}/HistoryComments Get transition comment history for multiple files Gets the comment histories for specified files that undergo the specified transition.
POST	api/{vaultName}/state/transitions POST: api/{vaultName}/state/transitions Get available transitions for multiple files Gets the transitions available for multiple files.

User

GET	api/{vaultName}/users/{userId} GET: api/{VaultName}/users/{UserID} or api/{VaultName}/users/{UserID}/info Get user info Gets information about the specified user.
GET	api/{vaultName}/users/{userId}/Extended GET: api/{VaultName}/users/{UserID}/Extended Get user info extended Gets extended information about the specified user.
PUT	api/{vaultName}/users/{userId}/Extended PUT: api/{VaultName}/users/{UserID}/Extended Set user info extended Saves information about the specified user.
GET	api/{vaultName}/users/{userId}/info GET: api/{VaultName}/users/{UserID} or api/{VaultName}/users/{UserID}/info Get user info Gets information about the specified user.
GET	api/{vaultName}/users/{userId}/Picture GET: api/{VaultName}/users/{UserID}/Picture Get user picture Gets the picture of the specified user.

PUT	api/{vaultName}/users/{userId}/Picture PUT: api/{VaultName}/users/{UserID}/Picture Put user picture Saves the picture of the specified user.
DELETE	api/{vaultName}/users/{userId}/Picture DELETE: api/{VaultName}/users/{UserID}/Picture Delete user picture Deletes the picture of the specified user.
GET	api/{vaultName}/users/all GET: api/{VaultName}/users or api/{VaultName}/users/all Get users Gets the users of the specified vault.

Vault

POST	api/{vaultName}/delete/computetree POST: api/{VaultName}/delete/computetree Delete compute tree Deletes the specified items in the specified vault.
GET	api/{vaultName}/info GET: api/{VaultName} or api/{VaultName}/info Get vault properties Gets the properties of the specified vault.

Version

Workflow

GET	api/{vaultName}/workflows/icons GET: api/{vaultName}/workflows/icons Get state icons Gets the workflow state icons in the specified vault.
-----	--

Search

POST	api/{vaultName}/search Gets objects in the vault that contain the specified string.
POST	api/{vaultName}/searchvariables Specifies parameters and variables for a search.

Web Event Hooks Programming Guide

- [Introduction](#)
- [Online resources](#)
- [Definitions](#)
- [What are the two types of web event hooks available with PDM Pro Web API?](#)
- [What are webhooks and why use them?](#)
- [How do I prepare to use webhooks with the PDM Pro Web API?](#)
- [What are the 18 event types supported by PDM Pro Web API?](#)
- [How do I set up webhooks for my vault?](#)
- [How do I retrieve the current webhooks in my vault?](#)
- [How do I activate webhooks for my vault?](#)
- [What are some advantages of custom add-in hooks?](#)
- [How do I use IAddInHook to create a custom add-in hook?](#)
- [How do I upload add-in hook DLLs to my vault?](#)
- [How do I retrieve a list of relative paths to add-in hook DLLs?](#)
- [How do I download a ZIP file containing all the add-in hook DLLs in my vault?](#)
- [How do I delete an add-in hook DLL from my vault?](#)
- [What are best coding practices for add-in hooks?](#)
- [What happens when a vault is deleted?](#)
- [How does the system handle hook cancellations?](#)
- [What is the file server root and how do I update it?](#)

Introduction

SOLIDWORKS PDM allows you to manage the revision control of enterprise file data, including bills of materials for SOLIDWORKS components and assemblies. The SOLIDWORKS PDM Pro API allows you to manage this data using your own custom applications written in VBA, .NET, Python, or JavaScript. You can program an add-in to handle events triggered in real time when vault changes occur.

The SOLIDWORKS PDM Pro Web API also provides the ability to manage your PDM vaults, instead using HTTP requests sent over the web from a PDM Pro Web API server. Now with SOLIDWORKS PDM Pro 2025 Web API, you can trigger events to occur when specific changes occur in the vault. A receiving application or web service provides a webhook URL that the webhook provider calls when a vault event happens. These so-called webhooks extend the PDM Pro Web API by providing the ability to notify registered customer applications in real time when PDM vault changes occur. Users can configure webhooks to cause events on one site (e.g., vault server) to trigger behavior on another (e.g., trigger builds or notify registered listener of changes). Additionally with PDM Pro 2025 Web API, you can create and upload to the web API server a custom add-in hook DLL that handles vault change events. This guide provides information about creating, configuring, and using PDM Pro Web API hooks, both add-in hooks and webhooks.

This manual uses Postman and JSON collections to demonstrate usage of webhooks. You can use other API platforms to see webhooks in action. See **PDM Pro Web API Help > Getting Started** to get started with Postman.

Online resources

The following links reference webhooks, the industry standard for messaging between online services:

<https://www.webhooks.org/>
<https://learn.microsoft.com/en-us/aspnet/webhooks/>
<https://docs.github.com/en/webhooks/about-webhooks>
<https://en.wikipedia.org/wiki/Webhook>

Definitions

- **Add-in hooks:** Custom dynamic link libraries developed by you to handle vault events. The main class implements the **SWPDM.Hooks.IAddInHook** interface, defining the **OnCmd** method to handle various events.
- **Webhooks:** User-defined HTTP callbacks triggered by specific vault events, allowing external web services to react in real-time.
- **Hooks helper:** A startup component responsible for initializing add-in hooks and webhooks.
- **Hooks provider:** A component responsible for loading, managing, and invoking add-in hooks and webhooks based on configured events.
- **Hooks customer:** An external web application or web service that registers its URL with the hooks provider to receive notifications.
- **Configuration management:** Storage and retrieval of settings related to add-in hooks, webhooks, and file server paths.
- **API endpoints:** RESTful interfaces that allow customers to manage add-in hooks and webhooks.
- **Security controls:** Mechanisms in place to protect the system, including authentication, authorization, input validation, and secure configuration management.

What are the two types of web event hooks available with PDM Pro Web API?

Webhooks (URL) - This is how a webhook provider sends HTTP requests when certain vault events occur. The URL address of a listening webhook customer application or web service.

Add-in hooks (DLL) - This is how you can program web events as you would PDM Pro API client application add-in events. In PDM Pro client add-in applications you implement **IEdmAddIn5::GetAddInInfo** and **IEdmAddIn5::OnCmd** to handle specific vault events. With a PDM Pro web api add-in, you

handle events by implementing `IAddInHook::OnCmd`. The DLL is uploaded to the web api server using one of the vault [configuration management API endpoints](#).

Both of these web event hooks can be used to:

- Extend web API capabilities by providing the ability to implement custom logic that automatically executes during vault operations, such as file addition, file check-in, or file state change.
- Integrate with external systems by connecting the PDM vault with external applications or services.
- Automate business processes by streamlining workflows and automating tasks like data validation, notifications, logging, or triggering external workflows when vault events occur.
- Enforce custom business rules by applying specific business logic and validations to ensure compliance and data integrity during vault operations.
- Customize the user experience by providing the ability to tailor workflows to better fit organization processes and user needs.

What are webhooks and why use them?

A webhook is a way of communicating and sharing information between online services in real time. Webhooks are HTTP requests, triggered by events in a source system, sent to a destination system, and often with a data payload. In order to process a webhook request, all events that can be triggered are registered, specifying all destinations. When an event is triggered, an HTTP request is sent to its corresponding destination.

Webhooks support:

- Configuration of HTTP endpoints that the system calls via HTTP POST when specific vault events occur.
- Sending data payloads to configured endpoints. Data payloads may contain detailed information about the event, vault name, and operation parameters such as Cancel.
- Returning specific HTTP status codes (e.g., 402 or 412) to cancel an operation.
- Configurable timeouts for webhook calls to ensure vault operations are not unreasonably delayed.
- API endpoints that get and set webhooks per vault, enabling dynamic management of webhook integration.

How do I prepare to use webhooks with the PDM Pro Web API?

1. A PDM Pro Web API server must be installed and configured for your vault using the Admin tool. See the PDM Pro installation documentation to learn how to do that.
2. Ensure that Internet Information Services (IIS) is installed on your computer.
3. Navigate to your PDM Pro Web API server root directory (e.g., `c:\inetpub\wwwroot\SOLIDWORKSPDMWebApi`).
Note: An alternative to steps 3 - 10 is step 23. Skip to step 11 and perform all remaining step instructions.
4. Open **vault.config.json** in any text editor. (You may need to change permissions to Admin if asked.)

```
{
  "Vaults": [
    {
      "Name": "vault_name",
      "DatabaseName": "database_name",
      "ArchiveServer": "archive_server_name",
      "ArchiveServerPort": 3030,
      "DatabaseServer": "database_server_name",
      "CommandTimeout": 200,
      "IndexingUserId": 0,
      "HooksTimeout": 15,
      "AddinHooksDlls": [
        "customHooksZip2\\customHooksZip2.dll",
        "customHooksZip\\customHooksZip.dll"
      ],
      "Webhooks": {
        "OnPreAddFolder": ["https://your_webhookURL"],
        "OnPostAddFolder": ["https://your_webhookURL1", "https://your_webhookURL2"],
        "OnPreAddFiles": [],
        "OnPostAddFiles": [],
        "OnPreGetFile": [],
        "OnPostGetFile": [],
        "OnPreLockFile": [],
        "OnPostLockFile": [],
        "OnPreUndoLockFile": [],
        "OnPostUndoLockFile": [],
        "OnPreCheckin": [],
        "OnPostCheckin": [],
        "OnPreChangeState": [],
        "OnPostChangeState": [],
        "OnPreDeleteFile": [],
        "OnPostDeleteFile": [],
        "OnPreDeleteFolder": [],
        "OnPostDeleteFolder": []
      }
    }
  ]
}
```

```

    }
  ],
  "FileServerRoot": "C:\\WINDOWS\\TEMP\\",
  "FolderCleanDelayHours": "24",
  "FolderCleanAgeHours": "24"
}

```

5. Edit the HooksTimeout variable in vault.config.json as needed for your webhooks only. (Note: The default timeout is 15 seconds. This variable does not apply to add-in hooks.)
6. Save **vault.config.json**.
7. In IIS, stop the web server site.
8. In IIS, stop the web server application pool.
9. In IIS, start the web server application pool.
10. In IIS, start the web server site.
11. In the Admin tool, log into your vault.
12. Click **Users and Groups Management > Users > Admin (System Administrator) > Administrative Permissions**.
13. Under Administrative permissions, ensure that **Can administrate add-ins** is selected.
14. Click **OK**.
15. Open Postman. See **PDM Pro Web API Help > Getting Started** to get started with Postman.
16. Load the **webapi environment JSON**. (The link to download this JSON file can be found in the **PDM Pro Web API help > Getting Started** topic.)
17. In Postman, ensure that the following environment variables are correctly set for your computer:
 - basicURL
 - host
 - vaultName
18. In Postman, send {{basicURL}}/authenticate to authenticate your session.
19. In Postman, copy the returned JwtString value to your environment variable, {{token}}.
20. Open **PDM Pro Web API Help > Getting Started**.
21. In the Event hooks section, click on HooksManagement.postman_collection.json and save it to your local disk.
22. Import **HooksManagement.postman_collection.json** into Postman by dragging and dropping the *.json file from your local disk to Postman.
23. (optional) Instead of steps 3-10 above, you can use the {{basicURL}}/configuration/hooks/url (POST) endpoint to update the webhooks in **vault.config.json**.
24. Open https://webhook.site to create a unique ID for testing webhooks. (You can also use https://beeceptor.com or any other tool that provides HTTP request debugging, intercepting, and proxying.)
25. In Postman modify each event in the Body of {{host}}api/{{vaultName}}/configuration/hooks/url (POST) to correspond to your webhook.site unique ID, for example, https://webhook.site/4ac246a9-7767-4597-bf7f-fb5704fb82f0.

What are the 18 event types supported by PDM Pro Web API?

PDM Pro Web API supports both pre- and post-operation events, giving you the ability to act or notify before or after a vault operation completes.

If this webhook is configured...	Then when this HTTP request is sent...	The webhook notifies your registered hooks customers...
OnPreAddFolder	{{basicURL}}/folders/:ParentFolderID (PUT)	Before a folder is added to the vault
OnPostAddFolder	{{basicURL}}/folders/:ParentFolderID (PUT)	After a folder is added to the vault
OnPreAddFiles	{{basicURL}}/files/:ChangeSetID/finishadd (PUT)	Before files are added to the vault
OnPostAddFiles	{{basicURL}}/files/:ChangeSetID/finishadd (PUT)	After files are added to the vault
OnPreGetFile	{{basicURL}}/files/:FileID/:Version/download (GET)	Before a file is downloaded from the vault
OnPostGetFile	{{basicURL}}/files/:FileID/:Version/download (GET)	After a file is downloaded from the vault
OnPreLockFile	{{basicURL}}/files/CheckOut (POST)	Before a file is checked out of the vault
OnPostLockFile	{{basicURL}}/files/CheckOut (POST)	After a file is checked out of the vault
OnPreUndoLockFile	{{basicURL}}/files/UndoCheckOut (POST)	Before an undo-checkout of a file
OnPostUndoLockFile	{{basicURL}}/files/UndoCheckOut (POST)	After an undo-checkout of a file
OnPreCheckin	{{basicURL}}/checkin/:ChangeSetID (PUT)	Before a file is checked into the vault
OnPostCheckin	{{basicURL}}/checkin/:ChangeSetID (PUT)	After a file is checked into the vault
OnPreChangeState	{{basicURL}}/state/:TransitionID/changestate (POST)	Before a file transitions to a new state
OnPostChangeState	{{basicURL}}/state/:TransitionID/changestate (POST)	After a file transitions to a new state
OnPreDeleteFile	{{basicURL}}/files/:FileID (DELETE)	Before a file is deleted from the vault
OnPostDeleteFile	{{basicURL}}/files/:FileID (DELETE)	After a file is deleted from the vault
OnPreDeleteFolder	{{basicURL}}/folders/:FolderID (DELETE)	Before a folder is deleted from the vault
OnPostDeleteFolder	{{basicURL}}/folders/:FolderID (DELETE)	After a folder is deleted from the vault

How do I set up webhooks for my vault?

To configure webhooks for your vault:

1. Open <https://webhook.site> to create a unique ID for testing webhooks. (You can also use <https://beeceptor.com> or any other tool that provides HTTP request debugging, intercepting, and proxying.)
2. In Postman's Collections left pane, navigate to **HooksManagement > SetWebHooks**.
3. The Body of `{{host}}api/{{vaultName}}/configuration/hooks/url` (POST) specifies webhook test site URLs that receive each operation/event type. Modify the unique ID for each to correspond to your webhook test site location.
4. In Postman, send `{{host}}api/{{vaultName}}/configuration/hooks/url` (POST) to register the webhooks.
5. Inspect the Response window of the HTTP request in Postman.
6. Confirm that the webhooks are updated in **vault.config.json** in your web server's root directory. Check the logs in **c:\ProgramData\SWPDM\WebAPI** for any errors that may have occurred during the update process.

How do I retrieve the current webhooks configured for my vault?

To retrieve the current webhooks configured for your vault:

1. In Postman's Collections left pane, navigate to **HooksManagement > GetWebHooks**.
2. In Postman, send `{{host}}api/{{vaultName}}/configuration/hooks/url` (GET).
3. Inspect the Response window of the HTTP request in Postman.

How do I activate webhooks configured for my vault?

To activate webhooks configured for your vault:

1. After you have configured webhooks and set up your testing website (<https://webhook.site> or <https://beeceptor.com>) to handle webhook events, test the webhooks by sending an HTTP request that triggers an event.
2. For example, send `{{basicURL}}/folders/:ParentFolderID` (PUT) to add a file to the vault.
3. If OnPreAddFiles and PostAddFiles webhook operations are configured, then you should see results in your testing website or registered URL.
4. Inspect your testing website or implementation to ensure the webhook worked. In webhook.site, each request displays the type of event received (SWPDM.Hooks HooksEventType enum):

Webhook operation	Event type
OnPreAddFolder	0
OnPostAddFolder	1
OnPreAddFiles	2
OnPostAddFiles	3
OnPreGetFile	4
OnPostGetFile	5
OnPreLockFile	6
OnPostLockFile	7
OnPreUndoLockFile	8
OnPostUndoLockFile	9
OnPreCheckin	10
OnPostCheckin	11
OnPreChangeState	12
OnPostChangeState	13

OnPreDeleteFile	14
OnPostDeleteFile	15
OnPreDeleteFolder	16
OnPostDeleteFolder	17

5. Check the logs in **c:\ProgramData\SWPDMWebAPI** for any errors during the webhook process.

6. After testing, modify the Body of `{{host}}api/{{vaultName}}/configuration/hooks/url` (POST) to add your own webhooks for each event; for example, replace the test webhook site URLs with your own website or other online application that is registered to respond to PDM vault changes. Send `{{host}}api/{{vaultName}}/configuration/hooks/url` (POST) to update the webhooks.

What are some advantages of custom add-in hooks?

If you regularly create PDM Pro client application add-ins, then you will appreciate the new ability to create custom add-in hooks with the PDM Pro 2025 Web API. The architecture is similar, and you can derive PDM Pro Web API add-in hooks using code you wrote for PDM Pro client application add-ins.

Add-in hooks support:

- **Dynamic Loading of Add-ins:** The system loads custom add-ins at runtime, isolating them in separate AppDomains to ensure stability and security.
- **Event Handling:** Add-ins can respond to a variety of vault events, such as pre/post file/folder additions, check-ins, check-outs, undo check-outs, file downloads, file/folder deletions, and state changes.
- **Operation Control:** Through the OperationParams type of OnCmd, add-ins can modify operation parameters and have the ability to cancel operations if necessary.
- **Management API Endpoints:** Provides RESTful API endpoints to upload, download, list, and delete add-in DLLs, facilitating easy deployment and management of custom add-ins.
- **Error Handling and Logging:** The system logs all significant events, errors, and exceptions during add-in and webhook operations, aiding in troubleshooting and ensuring operational transparency.

How do I use IAddInHook to create a custom add-in hook?

The PDM Pro Web API add-in hook is an add-in DLL that is programmed to implement IAddInHook. Your implementation of IAddInHook::OnCmd can handle events, control operations, handle errors, and perform logging.

The following custom add-in hook class uses SWPDM.Hooks.dll which contains the IAddInHook interface.

When OnCmd is called, it creates a text file with information about the triggered event (vault name, event type, operation parameters) and handles each type of triggered event.

```
using SWPDM.Hooks;
using System.IO;
using System;
using Newtonsoft.Json;
using Newtonsoft.Json.Serialization;
using ICSharpCode.SharpZipLib.Zip; // or other dependencies
using System.Threading;
using System.Collections.Generic;

namespace CustomHooksZip
{
    public class TestAddInHooks : MarshalByRefObject, IAddInHook
    {
        public void OnCmd(string vaultName, HooksEventType operationType,
            OperationParams operationParams)
        {
            // Define the directory and file paths
            string tempDirectory = Path.Combine(Path.GetTempPath(), "SWPDMHooks");
            Directory.CreateDirectory(tempDirectory);

            string fileName = $"{operationType}.txt";
            string filePath = Path.Combine(tempDirectory, fileName);

            string zipFileName = $"{operationType}.zip";
            string zipFilePath = Path.Combine(tempDirectory, zipFileName);

            try
            {
```

```

        string jsonContent = JsonConvert.SerializeObject(new { vaultName,
operationType, operationParams });

        // Write the content to a file
        File.WriteAllText(filePath, jsonContent);

        // Compress the file using ICSharpCode.SharpZipLib.Zip to test
dependencies
using (FileStream fs = File.Create(zipFilePath))
using (ZipOutputStream zipStream = new ZipOutputStream(fs))
{
    var entry = new ZipEntry(Path.GetFileName(filePath))
    {
        DateTime = DateTime.Now
    };
    zipStream.PutNextEntry(entry);
    using (FileStream fileStream = File.OpenRead(filePath))
    {
        fileStream.CopyTo(zipStream);
    }
    zipStream.CloseEntry();
}

// Handle specific operation types (if needed)
switch (operationType)
{
    case HooksEventType.OnPreAddFolder:
        var addFolderParams = operationParams as AddFolderParams;
        // Handle pre-add folder event
        break;
    case HooksEventType.OnPostAddFolder:
        addFolderParams = operationParams as AddFolderParams;
        // Handle post-add folder event
        break;
    case HooksEventType.OnPreAddFiles:
        var addFilesParams = operationParams as AddFilesParams;
        // Handle pre-add files event
        break;
    case HooksEventType.OnPostAddFiles:
        addFilesParams = operationParams as AddFilesParams;
        // Handle post-add files event
        break;
    case HooksEventType.OnPreCheckin:
        var checkInParams = operationParams as CheckInParams;
        // Handle pre-checkin event
        break;
    case HooksEventType.OnPostCheckin:
        checkInParams = operationParams as CheckInParams;
        // Handle post-checkin event
        break;
    case HooksEventType.OnPreChangeState:
        var changeStateParams = operationParams as ChangeStateParams;
        // Handle pre-change state event
        break;
    case HooksEventType.OnPostChangeState:
        changeStateParams = operationParams as ChangeStateParams;
        // Handle post-change state event
        break;
    case HooksEventType.OnPreGetFile:
        var getFileParams = operationParams as GetFileParams;
        // Handle pre-get file event
        break;
    case HooksEventType.OnPostGetFile:
        getFileParams = operationParams as GetFileParams;
        // Handle post-get file event

```

```

        break;
    case HooksEventType.OnPreDeleteFile:
        var deleteParams = operationParams as DeleteFileParams;
        // Handle pre-delete file event
        break;
    case HooksEventType.OnPostDeleteFile:
        deleteParams = operationParams as DeleteFileParams;
        // Handle post-delete file event
        break;
    case HooksEventType.OnPreDeleteFolder:
        var deleteFolderParams = operationParams as DeleteFolderParams;
        // Handle pre-delete folder event
        break;
    case HooksEventType.OnPostDeleteFolder:
        deleteFolderParams = operationParams as DeleteFolderParams;
        // Handle post-delete folder event
        break;
    case HooksEventType.OnPreLockFile:
        var lockFileParams = operationParams as LockFileParams;
        // Handle pre-checkout event
        break;
    case HooksEventType.OnPostLockFile:
        lockFileParams = operationParams as LockFileParams;
        // Handle post-checkout event
        break;
    case HooksEventType.OnPreUndoLockFile:
        var undoLockFileParams = operationParams as UndoLockFileParams;
        // Handle pre-undo checkout event
        break;
    case HooksEventType.OnPostUndoLockFile:
        undoLockFileParams = operationParams as UndoLockFileParams;
        // Handle post-undo checkout event
        break;

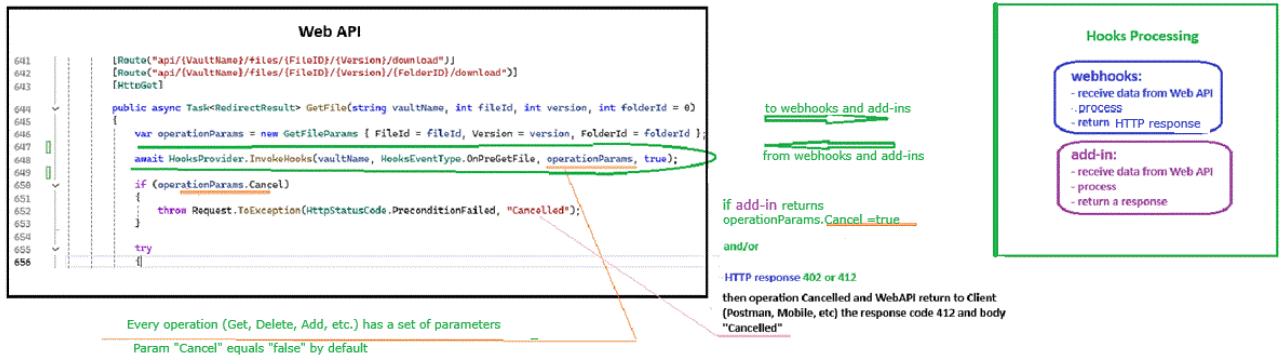
    default:
        // Handle default case if necessary
        break;
    }
}
catch (Exception ex)
{
    // Handle exceptions (e.g., log them)
    string logFilePath = Path.Combine(tempDirectory, "error.log");
    File.AppendAllText(logFilePath, $"{DateTime.Now}: Error in OnCmd -
{ex}\n");
}
}
}
}
}

```

You can use the code above to start building your add-in dynamic link library in Visual Studio:

1. Open Visual Studio.
2. Create a class library project.
3. Select Target Framework 4.5. (You may need to install this, depending on your version of Visual Studio.)
4. In **Tools > NuGet Package Manager > Manage NuGet Packages for Solution**, browse for and install **Newtonsoft.Json**. (This package is optional for logging. You can choose this or any other logging for your troubleshooting needs.)
5. In **Project > Add Project Reference > Browse**, add a reference to **SWPDM.Hooks.DLL**. This DLL can be found in the bin directory of your web api server (e.g., **c:\inetpub\wwwroot\SOLIDWORKSPDM\bin**).
6. Add optional references only if your add-in's functionality requires them. For example, if you want to compress a file as the example add-in hook above does, then you can add a reference to **ICSharpCode.SharpZipLib.DLL** in your project assemblies.
7. Copy the code above (TestAddInHooks) into your project's main class. The class inherits from **MarshalByRefObject** and implements the **IAddInHook** interface.
8. Implement **OnCmd(vaultName, operationType, operationParams)** to respond to various events. In **OnCmd**, you can also add logic to cancel operations when specific conditions occur (**operationParams.Cancel = true**). See [How does the system handle hook cancellations?](#) You can wrap your code in a try-catch block to handle any exceptions which prevents your add-in from crashing the host application. You can implement logging to help with debugging and monitoring. See [What are Best Coding Practices for Add-in Hooks?](#)
9. Build your project and locate the compiled DLL in your project's bin directory.

10. In Postman, upload the add-in's DLL(s) using `{{basicURL}}/configuration/hooks/dll` (PUT). (See the next section.)
11. Perform an operation that triggers a specific event (e.g., `OnPostGetFile`).
12. Monitor the hook customer application (e.g., `webhook.site`) to confirm that the add-in hook is invoked. You can customize how the URL responds by changing the response status code. To respond with a cancel code, edit the URL to respond with status code with 402 or 412. See [How does the system handle hook cancellations?](#) for more information.



13. In `TestAddInHooks`, a file is created on every pre- and post- event. Locate that file in `SWPDMHooks` in your file server root directory. Review logs in `c:\ProgramData\SWPDMWebAPI` for any messages related to the hook invocation and actions.

How do I upload add-in hook DLLs to my file server?

To upload add-in DLLs:

1. In Postman, authenticate your session using `{{basicURL}}/authenticate`.
2. In Postman's Collections left pane, navigate to **HooksManagement > UploadDll**.
3. Update the Authorization `{{token}}` with the token returned from your authentication.
4. Specify Header "X-Main-addin-DLL" with the name of your custom add-in main DLL.
5. In the Body tab of your request in Postman, select form-data. Add a key named "File" of type "File" (not "Text"). Click on the "Choose Files" button next to the key and select the DLL file(s) you want to upload.
6. Send `{{basicURL}}/configuration/hooks/dll` (PUT).
7. Inspect the Response window for the list of relative paths to the add-in DLLs (e.g., "customHooks\customHooks.dll").
8. Verify that the main DLL and its dependencies are uploaded to `file_server_root\storage\addins\vault_name\addin_folder_name`.
9. Ensure that the `vault.config.json` file's `AddinHooksDlls` section is updated with the relative path to the main DLL.

How do I retrieve a list of relative paths to add-in hook DLLs?

To get the list of relative paths to your add-in hook DLLs:

1. In Postman, authenticate your session using `{{basicURL}}/authenticate`.
2. In Postman's Collections left pane, navigate to **HooksManagement > GetHooksDllPaths**.
3. Update the Authorization `{{token}}` with the token returned from your authentication.
4. (optional) Specify Header "download" with a value of false (default).
5. Send `{{host}}api/{{vaultName}}/configuration/hooks/dll` (GET).
6. Inspect the Response window for the list of relative add-in DLL paths (e.g., "customHooks\customHooks.dll").

How do I download a ZIP file containing all the add-in hook DLLs configured for my vault?

To download a ZIP file containing all of the add-in hook DLLs configured for your vault:

1. In Postman, authenticate your session using `{{basicURL}}/authenticate`.
2. In Postman's Collections left pane, navigate to **HooksManagement > GetHooksDllFiles(download)**.
3. Update the Authorization `{{token}}` with the token returned from your authentication.
4. Specify Header "download" with a value of true.
5. Send `{{host}}api/{{vaultName}}/configuration/hooks/dll` (GET).
6. Locate the downloaded ZIP file in `file_server_root\storage\addins\vault_name\addin_folder_name`.

How do I delete an add-in hook DLL from my file server?

To delete an add-in hook DLL from my file server:

1. In Postman, authenticate your session using `{{basicURL}}/authenticate`.
2. In Postman's Collections left pane, navigate to **HooksManagement > DeleteAddin**.
3. Update the Authorization `{{token}}` with the token returned from your authentication.
4. Specify Header "X-Main-addin-DLL" with the name of your DLL.
5. In Postman, send `{{basicURL}}/configuration/hooks/dll` (DELETE).
6. Confirm that the DLL is deleted from `file_server_root\storage\addins\vault_name\addin_folder_name`.
7. Confirm that `vault.config.json` `AddinHooksDlls` section no longer specifies the add-in and all hooks are reloaded.

What are best coding practices for add-in hooks?

Add-in hooks are a powerful means of notifying interested parties about what's happening in your PDM vault. You must carefully design them. Your implementation of `IAddInHook::OnCmd()` is where the following best coding practices occur. Start with a basic add-in class as shown in the [How do I](#)

use [IAddInHook to create a custom add-in hook?](#) section, and then add the following code as you find appropriate to make your add-in hook code-safe and secure.

- **Logging of Hook Events and their Parameters**

In your add-in hook's OnCmd implementation, log hook events by capturing event details and writing them to log files. For auditing and debugging purposes, keep a record of all hook events triggered in the system.

- **Verifying File History when Pre-Check-In/Pre-ChangeState/etc. Events are Invoked**

Before a file is checked into the vault (or other event), check the file's history by making a WebAPI call through a 'service' user. Ensure that the file meets certain criteria or policies based on its history before allowing the check-in (or other events) to proceed.

- **Validating Mandatory Variable Before File Download**

Prior to allowing a file to be downloaded, verify whether a mandatory variable is set for the file (Author, Material, Revision etc). Make a WebAPI call through a 'service' user to perform this check, enforcing data completeness and compliance with business rules.

- **Preventing File Deletion When 'Destroy' Parameter Is True**

Prevent the deletion of files if the Destroy parameter is set to true, even when the user has permission to destroy files in PDM. Add an extra layer of protection to prevent accidental destruction of files via the WebAPI.

- **Notifying Users Upon File Lock**

After files are locked, send notifications (e.g., email, sms, or PDM notification via WebAPI) to relevant users. Ensure that stakeholders are informed about the status change of files.

- **Checking the 'Revoke' Parameter Before a State Change**

Before changing the state of a file, check whether the Revoke parameter is set to true.

- **Validating Folder Name Before Creation**

Check whether the folder name is null, empty, or does not contain a specific required word before allowing the folder to be added.

- **Logging of Folder Deletion Events**

When a folder is deleted, log the deletion event.

- **Post-processing a File after its Download**

After a file is retrieved (`{{basicURL}}/files/:FileID/:Version/download`), process the file by compressing it into a ZIP archive and copying it to a specified location. Automate post-download actions, such as archiving or distributing files.

- **Retrieving Variable Sets via SQL Connection**

Retrieve variable sets associated with a file by connecting directly to the SQL database. Provide custom data retrieval operations beyond the standard API capabilities.

- **Checking for Active BOMs Before Folder Deletion via SQL Connection**

Before a folder is deleted, check through an SQL connection whether the folder contains any active or undeleted BOMs. If active BOMs are found, prevent deletion, ensuring critical data is not lost.

What happens when a vault is deleted?

Add-in hooks and the authentication token are still alive when the vault is deleted from **vault.config.json**. Add-in hooks that were uploaded for the deleted vault get removed along with their folders on the next startup or restart.

How does the system handle hook cancellations?

1. If a pre-event hook is configured for an operation, then when that operation is about to start, the WebAPI passes parameters, including the cancel flag, to all configured webhook sites and/or add-in hooks.
2. Some logic is executed in the add-in and/or on the webhook site. Based on that logic, the webhook site and/or add-in responds to the WebAPI with whether the operation should be cancelled (Cancel=TRUE) or not (Cancel=FALSE).
3. The WebAPI receives the response from the add-in or web service. If the response is Cancel=TRUE, then the WebAPI proceeds to notify all add-ins and webhooks with a payload that contains Cancel=TRUE. If the WebAPI receives response Cancel=FALSE, then it notifies all add-ins and webhooks with a payload that contains Cancel=FALSE.

When multiple add-ins and/or webhooks are configured for a vault operation, the system handles cancellations gracefully to ensure all parties are properly notified.

Sequential Notification: If any add-in or webhook requests to cancel an operation, the system does not terminate the operation immediately. Instead, it continues to notify all add-ins and webhooks in the configured list about the cancellation.

Final Cancellation: Only after all add-ins and webhooks have been notified does the system proceed to cancel the operation.

The [table](#) below contains three scenarios for pre-event hook cancellations. In each scenario, five pre-event hooks are configured for an operation: two add-in hooks and three webhooks. The add-in hooks are configured to be sent sequentially (in **vault.config.json**), and the webhooks are sent in parallel. The table shows how cancellations are handled.

Pre-event example 1

Webhook#3 does not know that webhook#2 responded to the WebAPI with Cancel=TRUE. Webhook#3 still responds with Cancel=FALSE because it has met the conditions in the webhook#3 logic. Webhook #3 is unaware of webhook#2's cancellation. The overall operation is cancelled, and the WebAPI returns 412/Cancelled.

Pre-event example 2

Add-in #2 responds to the WebAPI with Cancel=TRUE. All of the webhooks receive data payloads containing Cancel=TRUE (the add-in cancelled, and web services are notified). The overall operation is cancelled, and the WebAPI returns 412/Cancelled.

Pre-event example 3

Webhook#2 responds to the WebAPI with Cancel=TRUE, but it times out, effectively negating the cancellation. The overall operation succeeds, and the WebAPI returns 200 or another return code, depending on conditions.

Notes:

- The payloads/data for some webhooks may not be up to date if one of the webhooks responds with "402 or 412 (WebAPI interprets these as "Cancel=TRUE"). Because all webhooks are sent in parallel, the cancellation by one webhook may not be represented in other concurrent webhooks.
- Beware of webhook timeout effects (e.g., Pre-event example 3). If one webhook flags to cancel the operation but times out, the cancellation is negated and the overall operation succeeds.

Logic sequence:	Time slice #1	Time slice #2	Time slice #3	Later time slices
Pre-event example 1:				
add-in #1	- WebAPI sends data with default flag Cancel=FALSE to add-in - add-in executes logic, for example, checks for file variable	add-in returns Cancel=FALSE (i.e., file variable exists) to WebAPI		
add-in #2		- WebAPI sends data with default flag Cancel=FALSE to add-in - add-in executes logic, for example, checks for file history	add-in returns Cancel=FALSE (i.e., file history ok) to WebAPI	
After all add-ins execute sequentially, WebAPI sends all webhooks in parallel				
webhook#1			WebAPI sends data to web service with default Cancel = FALSE	Web service returns 200, and WebAPI interprets this as Cancel = FALSE
webhook#2			WebAPI sends data to web service with default Cancel = FALSE	Web service returns 402 or 412, and WebAPI interprets this as Cancel = TRUE (i.e. conditions not met, so cancel)
webhook#3			WebAPI sends data to web service with default Cancel = FALSE	Web service returns 200, and WebAPI interprets this as Cancel = FALSE
Overall WebAPI operation cancelled , WebAPI returns status code 412 and response Body "Cancelled" Note: Since all webhooks are sent in parallel, when webhook#3 sends Cancel = FALSE , it has no idea webhook#2 sent Cancel=TRUE back to the WebAPI				
Pre-event example 2:				
add-in #1	- WebAPI sends data with default flag Cancel=FALSE to add-in - add-in executes logic, for example, checks for file variable	add-in returns Cancel=FALSE (i.e., file variable exists) to WebAPI		
add-in #2		- WebAPI sends data with default flag Cancel=FALSE to add-in - add-in executes logic, for example, checks for file history	add-in returns Cancel=TRUE (i.e., missing file history, so send Cancel) to WebAPI	
After all add-ins execute sequentially, WebAPI sends all webhooks in parallel				
webhook#1			WebAPI sends data with default flag Cancel=TRUE	Web service returns 200, and WebAPI interprets this as no need to cancel, but cancel flag was already set to true because previous add-in's logic returned Cancel=TRUE
webhook#2			WebAPI sends data with default flag Cancel=TRUE	Web service returns Cancel=TRUE

webhook#3		WebAPI sends data with default flag Cancel=TRUE	Web service returns Cancel=TRUE
Overall WebAPI operation cancelled ; WebAPI returns status code 412 and response body "Cancelled"			
Pre-event example 3:			
add-in #1	- WebAPI sends data with default flag Cancel=FALSE to add-in - add-in executes logic, for example, checks for file variable	add-in returns Cancel=FALSE (i.e. file variable exists) to WebAPI	
add-in #2		- WebAPI sends data with default flag Cancel=FALSE to add-in - add-in executes logic, for example, checks for file history	add-in returns Cancel=FALSE , (i.e., file history ok) to WebAPI
After all add-ins execute sequentially; WebAPI sends all webhooks in parallel			
webhook#1		WebAPI sends data with default flag Cancel=FALSE	Web service returns 200, and WebAPI interprets this as no need to cancel; Cancel=FALSE ; response time 1 sec
webhook#2		WebAPI sends data with default flag Cancel=FALSE	Web service returns 402 or 412, WebAPI interprets this as cancel, but the response time is 18 sec, which is over the 15 sec timeout setting , so the effective response is Cancel=FALSE
webhook#3		WebAPI sends data with default flag Cancel=FALSE	Web service returns Cancel=FALSE
Overall WebAPI operation completed ; WebAPI returns status code 200,403, or other, depending on conditions; Post-events, if configured, execute			

What is the file server root and how do I update it?

This note pertains to IIS operational behavior and is not directly related to event hooks. The file server root is where your add-in hook DLLs reside. The default file server root with PDM Pro Web API server installation is **c:\windows\temp**. Your add-ins are stored in the **storage/addins/vault_name/addin_folder_name** subdirectory of the file server root. When you update the file server root path, your add-ins are migrated to the new location, **vault.config.json** is updated, and webhooks are reloaded.

1. Open **PDM Pro Web API Help > Getting Started**.
2. In the Event hooks section, click on **GetPostFileServerRoot.postman_collection.json** and save it to your local disk.
3. Import **GetPostFileServerRoot.postman_collection.json** into Postman by dragging and dropping the *.json file from your local disk to Postman.
4. In Postman's Collections left pane, navigate to **GetPostFileServerRoot > GetFileServerRoot**.
5. Send **{{host}}api/configuration/fileserverroot (GET)**.
6. Inspect the Response window of the HTTP request.
7. To update the file server root, in Postman's Collections left pane, navigate to **GetPostFileServerRoot > PutFileServerRoot**.
8. Modify the Body of **{{host}}api/configuration/fileserverroot (POST)** to change the file server root location.
9. Send **{{host}}api/configuration/fileserverroot (POST)**.
10. Inspect the Response window.

NOTE: Is is strongly recommended that you restart IIS to prevent potential conflicts. Without a restart, IIS might temporarily continue using the previous file server root path due to recycling delays.

ReleaseNotes (Topic)

PDM Pro Web API Help

Release Notes

Glossary Item Box

- [SOLIDWORKS PDM Professional 2025 Web Api](#)
- [SOLIDWORKS PDM Professional 2024 Web API](#)
- [SOLIDWORKS PDM Professional 2023 Web API](#)
- [SOLIDWORKS PDM Professional 2022 Web API](#)

SOLIDWORKS PDM Professional 2025 Web API

Configuration

- [api/configuration/fileserverroot \(Get\)](#)
- [api/configuration/fileserverroot \(Post\)](#)
- [api/{vaultName}/configuration/hooks/dll \(Get\)](#)
- [api/{vaultName}/configuration/hooks/dll \(Put\)](#)
- [api/{vaultName}/configuration/hooks/dll \(Delete\)](#)
- [api/{vaultName}/configuration/hooks/url \(Get\)](#)
- [api/{vaultName}/configuration/hooks/url \(Post\)](#)

SOLIDWORKS PDM Professional 2024 Web API

Data Card

- [api/{vaultName}/files/{fileId}/{folderId}/datacard \(Post\)](#)
- [api/{vaultName}/files/{fileId}/datacard?folderId={folderId} \(Post\)](#)

File

- [api/{vaultName}/files/{fileId}?folderId={folderId}&destroy={destroy} \(Delete\)](#)
- [api/{vaultName}/files/{fileId}/{folderId}?destroy={destroy} \(Delete\)](#)
- [api/{vaultName}/files/{fileId}/{version}/configurations?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/configurations \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/info/extended?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/info/extended \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/ActiveConfig?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/ActiveConfig \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/variables?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/variables \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/versions \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/allreferences?configId={configId}&includeSubParents={includeSubParents} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/references?configId={configId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}?version={version}&folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/info?version={version}&folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/info?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/info \(Get\)](#)
- [api/{vaultName}/files/{fileId}/transitions?version={version}&folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/transitions?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/transitions \(Get\)](#)
- [api/{vaultName}/files/{fileId}/moves?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{folderId}/moves \(Get\)](#)
- [api/{vaultName}/files/{fileId}/history?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{folderId}/history \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{configId}/whereused?folderId={folderId}&anyVersion={anyVersion} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{configId}/{folderId}/whereused?anyVersion={anyVersion} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{folderId}/bominfo \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/download?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/download \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/thumbnails?folderId={folderId} \(Get\)](#)
- [api/{vaultName}/files/{fileId}/{version}/{folderId}/thumbnails \(Get\)](#)

Search

- [api/{vaultName}/search \(Get\)](#)
- [api/{vaultName}/search \(Post\)](#)
- [api/{vaultName}/searchvariables \(Post\)](#)

Stage

- [api/stage/{vaultName}/{documentId}?folderId={folderId}&overrideVersion={overrideVersion}](#) (Post)

State Transition

- [api/{vaultName}/state/{documentId}/transitions?folderId={folderId}](#) (Get)
- [api/{vaultName}/state/{documentId}/{folderId}/transitions](#) (Get)

SOLIDWORKS PDM Professional 2023 Web API

File

- [api/{vaultName}/files/infofrompath](#) (Post)

SOLIDWORKS PDM Professional 2022 Web API

Authentication

- [api/{vaultName}/authenticate](#) (Post)

BOM

- [api/{vaultName}/bom/{bomDocumentId}/{version}/{folderId}/named](#) (Get)
- [api/{vaultName}/bom/{bomTypeId}/{fileId}/{version}/{folderId}/computed?configId={configId}&latest={latest}](#) (Get)
- [api/{vaultName}/bom/{bomTypeId}/{fileId}/{version}/{folderId}/weldmentcutlist?configId={configId}&latest={latest}](#) (Get)

Configuration

- [api/configuration/vaults?vaultName={vaultName}](#) (Get)

Data Card

- [api/{vaultName}/files/{fileId}/{version}/datacard?folderId={folderId}](#) (Get)
- [api/{vaultName}/files/{fileId}/datacard](#) (Post)
- [api/{vaultName}/folders/{folderId}/datacard](#) (Get)

Default

- [api/{uri}](#) (Get)
- [api/{uri}](#) (Post)
- [api/{uri}](#) (Put)
- [api/{uri}](#) (Patch)
- [api/{uri}](#) (Delete)

File

- [api/{vaultName}/files/{fileId}/{version}](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/ActiveConfig](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/allreferences?configId={configId}&folderId={folderId}&includeSubParents={includeSubParents}](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/configurations](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/download](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/info](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/info/extended](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/references?configId={configId}&folderId={folderId}](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/thumbnails](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/variables](#) (Get)
- [api/{vaultName}/files/{fileId}/{version}/whereused?configId={configId}&folderId={folderId}&anyVersion={anyVersion}](#) (Get)
- [api/{vaultName}/files/{fileId}/bominfo?folderId={folderId}](#) (Get)
- [api/{vaultName}/files/{fileId}/info?version={version}](#) (Get)
- [api/{vaultName}/files/{fileId}/transitions?version={version}](#) (Get)
- [api/{vaultName}/files/{fileId}/versions?folderId={folderId}](#) (Get)
- [api/{vaultName}/files/{fileId}?destroy={destroy}](#) (Delete)
- [api/{vaultName}/files/{fileId}?version={version}](#) (Get)

- [api/{vaultName}/files/buildtree/checkout \(Post\)](#)
- [api/{vaultName}/files/buildtree/undo \(Post\)](#)
- [api/{vaultName}/files/CheckOut \(Post\)](#)
- [api/{vaultName}/files/info \(Post\)](#)
- [api/{vaultName}/files/UndoCheckOut \(Post\)](#)

Folder

- [api/{vaultName}/folders/{folderId} \(Get\)](#)
- [api/{vaultName}/folders/{folderId}/browse \(Get\)](#)
- [api/{vaultName}/folders/{folderId}/info \(Get\)](#)
- [api/{vaultName}/folders/{folderId}?destroy={destroy} \(Delete\)](#)
- [api/{vaultName}/folders/{parentFolderId} \(Put\)](#)
- [api/{vaultName}/folders/info \(Post\)](#)

Group

- [api/{vaultName}/groups \(Get\)](#)
- [api/{vaultName}/groups/{groupId} \(Get\)](#)
- [api/{vaultName}/groups/{groupId}/info \(Get\)](#)
- [api/{vaultName}/groups/all \(Get\)](#)

Notification

- [api/{vaultName}/notifications \(Post\)](#)
- [api/{vaultName}/notifications/markRead \(Post\)](#)
- [api/{vaultName}/notifications?all={all}&pageNo={pageNo}&pageSize={pageSize} \(Get\)](#)

Progress

- [api/{vaultName}/progress/{guid}/result \(Get\)](#)
- [api/{vaultName}/progress/{guid}/status \(Get\)](#)

Stage

- [api/{vaultName}/changeset/create \(Get\)](#)
- [api/{vaultName}/checkin/{changesetId}/{overrideVersion} \(Put\)](#)
- [api/{vaultName}/checkin/addfiles/{changesetId} \(Put\)](#)
- [api/{vaultName}/checkin/buildtree/{changesetId} \(Get\)](#)
- [api/{vaultName}/files/{changesetId}/finishadd \(Put\)](#)
- [api/{vaultName}/files/{changesetId}/upload \(Put\)](#)

State Transition

- [api/{vaultName}/state/{documentId}/{folderId}/{transitionId}/references \(Get\)](#)
- [api/{vaultName}/state/{documentId}/transitions \(Get\)](#)
- [api/{vaultName}/state/{transitionId} \(Get\)](#)
- [api/{vaultName}/state/{transitionId}/changestate?revoke={revoke} \(Post\)](#)
- [api/{vaultName}/state/{transitionId}/DynamicNotificationUsers \(Post\)](#)
- [api/{vaultName}/state/{transitionId}/HistoryComments \(Post\)](#)
- [api/{vaultName}/state/transitions \(Post\)](#)

User

- [api/{vaultName}/users \(Get\)](#)
- [api/{vaultName}/users/{userId} \(Get\)](#)
- [api/{vaultName}/users/{userId}/Extended \(Get\)](#)
- [api/{vaultName}/users/{userId}/Extended \(Put\)](#)
- [api/{vaultName}/users/{userId}/info \(Get\)](#)
- [api/{vaultName}/users/{userId}/Picture \(Get\)](#)
- [api/{vaultName}/users/{userId}/Picture \(Put\)](#)
- [api/{vaultName}/users/{userId}/Picture \(Delete\)](#)
- [api/{vaultName}/users/all \(Get\)](#)

Vault

- `api/{vaultName}` (Get)
- `api/{vaultName}/delete/computetree` (Post)
- `api/{vaultName}/info` (Get)
- `api/{vaultName}/search` (Get)
- `api/{vaultName}/search` (Post)

Version

- `api/version/webapi` (Get)

Workflow

- `api/{vaultName}/workflows` (Get)
- `api/{vaultName}/workflows/icons` (Get)

Webhooks (Topic)

PDM Pro Web API Help

Webhooks

Glossary Item Box

As of 2025, PDM Pro Web API provides support for configuring and handling vault events and managing custom hook add-in DLLs. Click and save the following JSON collections to your computer. Then drag/drop them into Postman:

[HooksManagement.postman_collection.json](#)

[GetPostFileServerRoot.postman_collection.json](#)

For details, see the [Web Event Hooks Programming Guide](#).

Welcome (Topic)

PDM Pro Web API Help

Welcome

Glossary Item Box

The SOLIDWORKS PDM Professional Web API:

- allows you to build Representation State Transfer (REST-ful) web applications using SOLIDWORKS PDM Professional.
- consists of controller APIs that manage various product data operations from your web site.

See the [Getting Started](#) and [Release Notes](#) topics.